CLASS 15+16 HOMEWORK Lists, Data Fetching & AI Tools

XELI AI Labs • Frontend Engineering • Due: Next Class

4–5 hours estimated	100 points total	AI tools encouraged!

■ A Note on This Homework
Two projects this time — not five. We want you to build two things well, not five things quickly. Use AI tools to help. But understand what you submit.

PROJECT 1 Todo List App 50 pts

■ In Plain English
A todo list sounds simple. It is. But it uses EVERY React skill you've learned: useState, controlled inputs, .map() for the list, and conditional rendering for the empty state and the done/not-done styles. This is your proof that you understand React.

STATE VARIABLES

`todos` `array` — List of todo objects: { id, text, done }. Initial: []

`input` `string` — What the user is currently typing. Initial: ''

REQUIREMENTS

- ■ File: src/components/TodoList.jsx
- ■ Text input + Add button to create new todos
- ■ The todo list rendered with .map() — one element per todo
- ■ Click a todo to toggle it done/not done (show strikethrough when done)
- ■ A delete button (X) on each todo to remove it from the list
- ■ Show "X tasks remaining" count (only when the list is not empty)
- ■ A "Clear completed" button that removes all done todos
- ■ An encouraging empty state message when the list has zero todos
- ■ Press Enter in the input to add (not just the button)

STARTER HINT

```
const [todos, setTodos] = useState([]); // ADD — never mutate! Create new array: const addTodo = () => { setTodos(prev => [...prev, { id: Date.now(), text: input, done: false }]); setInput(''); }; // TOGGLE — .map() through, flip the matching one: const toggleTodo = (id) => { setTodos(prev => prev.map(t => t.id === id ? {...t, done: !t.done} : t)); }; // DELETE — .filter() keeps everything EXCEPT the deleted one: const deleteTodo = (id) => { setTodos(prev => prev.filter(t => t.id !== id)); };
```

AI TIPS FOR THIS PROJECT

■ If stuck on toggle: "Explain how to toggle a boolean field in a React state array without mutating the original array."

■ If stuck on delete: "How do I remove an item from a React state array using .filter()? Explain why we use filter instead of splice."

BONUS POINTS

★ Filter buttons: All / Active / Completed — show only matching todos

★ Drag to reorder (use a library like @dnd-kit/core)

PROJECT 2 Live Data App 50 pts

■ In Plain English
Fetch real data from the internet and display it. You pick the API — start simple (jokes), then try something more interesting if you finish early. The fetch pattern is always the same. Just swap the URL.

STATE VARIABLES (same every time)

- `data` `Initial: [] or null` — The fetched data. Use [] for lists, null for single items
- `loading` `Initial: true` — Start true — spinner shows immediately
- `error` `Initial: null` — Null until something goes wrong

REQUIREMENTS

- ■ File: src/components/LiveDataApp.jsx
- ■ Fetch from ANY of the free APIs listed below (or find your own)
- ■ Show a loading state ("Loading..." or a spinner) while fetching
- ■ Show an error message if the fetch fails (with a "Try again" button)
- ■ Show an empty state if the API returns no results
- ■ Display the fetched data as a styled list or grid
- ■ A "Refresh / Get New" button that fetches fresh data without page reload
- ■ Minimum 3 fields from the API displayed per item

FREE APIS — NO KEY REQUIRED

API	URL	What you get
Random Joke	`official-joke-api.appspot.com/random_joke`	setup, punchline (great starter!)
Random Users	`randomuser.me/api/?results=6`	photo, name, email, city
Random Quote	`quotable.io/random`	content, author, tags
Products	`fakestoreapi.com/products`	title, price, image, rating
Countries	`restcountries.com/v3.1/all`	name, capital, population, flag
Cat Facts	`catfact.ninja/fact`	fact, length

STARTER TEMPLATE — COPY THIS

```
import { useState, useEffect } from 'react'; const LiveDataApp = () => { const [data, setData] = useState([]); const [loading, setLoading] = useState(true); const [error, setError] = useState(null); const fetchData = async () => { setLoading(true); setError(null); try { const res = await fetch('YOUR_URL_HERE'); const json = await res.json(); setData(json); // adjust based on API structure } catch { setError('Could not load data.'); } finally { setLoading(false); } }; useEffect(() => { fetchData(); }, []); if (loading) return <p>Loading...</p>; if (error) return <p>{error} <button onClick={fetchData}>Retry</button></p>; return ( <div> <button onClick={fetchData}>Refresh</button> {/* render your data here with .map() */} </div> ); }; export default LiveDataApp;
```

AI TIPS FOR THIS PROJECT

■ Paste the raw API JSON into Claude and say: "What fields should I use to display a nice user card from this response? Show me example JSX."

■ If fetch fails: "I'm getting a network error when I fetch from [URL] in React. What are the most common reasons this fails and how do I debug it?"

BONUS POINTS

★ Display the fetch timestamp: "Last updated: 3 minutes ago"

★ Auto-refresh every 30 seconds using setInterval inside useEffect

---

Grading Rubric

Project	What We Look For	Pts
Todo List	useState · .map() list · toggle · delete · conditional count · empty state	50
Live Data App	useEffect · fetch() · loading state · error state · render list · refresh button	50

Remember: Use AI tools. Understand what you submit. Two simple projects built well beat five half-finished ones.